WEEKASH

Handoff técnico y contexto consolidado para continuar el desarrollo en Codex

Objetivo: que una nueva sesión/agente pueda continuar Weekash sin depender del historial de ChatGPT y sin confundir decisiones reales con propuestas futuras.

Etiqueta	Significado
CONFIRMADO	Dato recuperado del trabajo previo o del estado conocido del proyecto.
DECIDIDO / PENDIENTE	Requisito acordado, pero no se afirma que esté implementado.
CONTEXTO COMPLETADO	Especificación añadida para hacer el handoff operativo; debe validarse contra el repositorio antes

1. Resumen ejecutivo

CONFIRMADO. Weekash es una app personal de finanzas enfocada en controlar el dinero disponible durante la semana. La experiencia principal gira alrededor de un presupuesto semanal, el dinero restante, los gastos registrados y una interfaz móvil muy visual y rápida.

CONFIRMADO. La dirección técnica de Weekash V2 es React Native + Expo, con store global, persistencia local, arquitectura por capas, sistema de tema/tokens y build Android. El documento ARQUITECTURA.md se estableció como guía principal obligatoria; FINANZAS_APP.md y la app antigua solo son referencias.

REGLA DE PROYECTO. Trabajar por fases y no avanzar a la siguiente hasta cerrar la actual. Codex debe inspeccionar el repositorio antes de asumir qué parte de estas fases está realmente terminada.

2. Stack y entorno conocido

• CONFIRMADO: React Native + Expo.

• CONFIRMADO: Expo Router figura en el contexto técnico posterior del proyecto.

• CONTEXTO HISTÓRICO: se han manejado Expo SDK 53/54 y React Native 0.81.5. Codex debe leer package.json y lockfile y tomar el repositorio como autoridad si las versiones difieren.

• CONFIRMADO: desarrollo/build Android local con Android Studio.

• CONTEXTO HISTÓRICO: Java 21 se ha usado en el entorno de desarrollo. Validar compatibilidad con Gradle/Expo antes de cambiar nada.

• CONFIRMADO: persistencia local y store global forman parte de la arquitectura.

• CONFIRMADO: sistema de theme/tokens para evitar estilos dispersos.

No migrar versiones, librerías, navegación ni almacenamiento únicamente porque exista una versión más nueva. Primero hacer inventario del proyecto actual y conservar lo que ya funcione.

3. Estructura arquitectónica acordada

CONFIRMADO. La estructura base recuperada es:

```
src/ app/ navigation/ screens/ components/ store/ theme/ utils/ types/ storage/ docs/
```

La intención es mantener responsabilidades separadas: pantallas para composición, componentes reutilizables para UI, store para estado de negocio, storage para persistencia, theme para tokens

visuales, utils para lógica pura y types para contratos de datos.

CONTEXTO COMPLETADO: si Expo Router ya está plenamente adoptado en el repo, `src/app` puede asumir routing y `navigation` quedar solo para helpers/configuración. No duplicar dos sistemas de navegación.

4. Fuente de verdad y prioridad documental

Prioridad	Fuente	Uso
1	Código actual + package.json/lockfile	Estado real e implementación existente.
2	ARQUITECTURA.md	Decisiones obligatorias de arquitectura y fases.
3	Este handoff	Contexto consolidado, requisitos y continuidad para Codex.
4	FINANZAS_APP.md	Referencia funcional/histórica.
5	App antigua	Inspiración y recuperación de ideas; no copiar deuda técnica.

5. Home: diseño y comportamiento acordados

CONFIRMADO. La Home debe ser funcional y directa, no explicativa. Se pidió eliminar textos sobrantes y evitar llenar la pantalla de tarjetas.

• Presupuesto semanal visible y editable mediante un icono de lápiz verde.

• Dinero restante como foco visual principal, asociado a un círculo/indicador central.

• Texto de progreso tipo “Has usado X de Y”.

• Bloque de “Semana actual” con los días de lunes a domingo.

• Píldoras de días modernas, visualmente limpias y coherentes con el resto de la interfaz.

• Una única tarjeta principal para “Gastos del día”, en lugar de múltiples cards explicativas.

• Acción clara para añadir gasto mediante botón/FAB “+”.

• Panel/acciones principales recuperadas del diseño: Semana / Gastos / Ajustar.

• Eliminar el texto “Movimiento manual” o, si sigue siendo necesario, separarlo visualmente del importe para que no compita con el dato principal.

Semántica de consumo del presupuesto

Porcentaje usado	Estado visual
0-49 %	Verde
50-74 %	Amarillo
75-89 %	Naranja
90 % o más	Rojo

CONTEXTO COMPLETADO: interpretar estos umbrales como porcentaje gastado del presupuesto semanal, porque el cambio de verde a rojo representa aumento de riesgo. Centralizar la función que calcula porcentaje/color para que Home, calendario y estadísticas no diverjan.

6. Navegación y lenguaje visual

• CONFIRMADO: navegación inferior transparente/tipo cristal (glass), ligera y sin aspecto de barra pesada.

• CONFIRMADO: iconos minimalistas verdes.

• CONFIRMADO: destinos planteados: Home, Estadísticas, Añadir, Calendario y Ajustes.

• CONFIRMADO: paleta verde/crema asociada al proyecto. Referencia visual histórica: verde oscuro aproximado #1f4d3a, crema #f5f3ea y beige #e8dfc9; validar tokens reales antes de reemplazar colores existentes.

• CONFIRMADO: icono y splash forman parte del acabado del producto.

La intención visual es que Weekash parezca una app cuidada y moderna, pero sin sacrificar lectura rápida. La Home debe responder primero a: cuánto me queda, cuánto he gastado, en qué día estoy y cómo añado un gasto.

7. Modelo funcional mínimo

CONTEXTO COMPLETADO para Codex. Este modelo formaliza lo ya decidido sin afirmar que estos nombres existan en el código:

> WeeklyBudget - amount: number - weekStart: ISO date - weekEnd: ISO date Expense - id: string - amount: number - date: ISO datetime/date - category?: string - note?: string - createdAt: ISO datetime Derived - spent = suma de gastos de la semana - remaining = budget.amount - spent - usedPct = budget.amount > 0 ? spent / budget.amount * 100 : 0

Reglas recomendadas: importes monetarios no deben depender de floats sin control; conservar una estrategia consistente (por ejemplo céntimos enteros) si el código ya la usa. La semana y la zona horaria deben calcularse de una sola forma en toda la app.

8. Roadmap/fases recuperadas

Fase	Alcance
1. Base	Expo, navegación, store, tema/tokens, persistencia y capacidad de build Android.
2. Home	Presupuesto semanal, restante, progreso, semana/días, gastos del día y alta de gasto.
3. Pulido	Correcciones visuales/UX de Home y consistencia del diseño.
4. Calendario	Consulta temporal de gastos y navegación por fechas.
5. Compartidos	Gastos compartidos.
6. Ajustes	Configuración de la app y del comportamiento financiero.
7. QA	Pruebas funcionales, estados límite, persistencia, navegación y regresiones.
8. Build Android	APK/build estable para uso real.

Regla: no saltar de fase por entusiasmo. Antes de abrir Calendario/Compartidos, cerrar los criterios de aceptación de Home y su persistencia.

9. Funcionalidades posteriores ya contempladas

• DECIDIDO / PENDIENTE: Calendario.

• DECIDIDO / PENDIENTE: Gastos compartidos.

• DECIDIDO / PENDIENTE: Resumen diario.

• DECIDIDO / PENDIENTE: Estadísticas (ya aparece como destino de navegación).

• EVOLUCIÓN OPCIONAL, NO MVP CONFIRMADO: versión web, login, base de datos remota, panel admin y despliegue cloud fueron propuestas para convertir Weekash en proyecto de portfolio. No introducirlas en la app móvil actual sin una decisión explícita.

10. Especificación de continuidad para Codex

Al abrir el repositorio, Codex debería seguir este orden:

• Leer `ARQUITECTURA.md` completo y después `package.json`, lockfile, configuración Expo y estructura real de `src/`.

• Comparar el código con este handoff y crear una lista breve: implementado / parcialmente implementado / pendiente / contradicción.

• Ejecutar instalación, typecheck/lint/tests disponibles y arrancar Expo antes de modificar arquitectura.

• Verificar que el store persiste correctamente presupuesto y gastos tras reiniciar la app.

• Revisar Home contra los requisitos visuales de la sección 5.

• Corregir primero errores que bloqueen build o uso básico; después cerrar la fase activa.

• No reescribir componentes funcionales solo por preferencia estética del agente.

• Antes de una migración de SDK/RN, justificarla por un problema real y comprobar compatibilidad de dependencias.

11. Criterios de aceptación sugeridos para cerrar Home

CONTEXTO COMPLETADO. Estos criterios convierten las decisiones anteriores en pruebas concretas:

• Se puede definir/editar un presupuesto semanal y permanece tras cerrar/reabrir la app.

• El restante y “Has usado X de Y” se recalculan inmediatamente al añadir/eliminar/modificar un gasto.

• Los umbrales de color cambian exactamente en 50 %, 75 % y 90 % de gasto.

• La semana actual muestra correctamente lunes-domingo y el día seleccionado.

• “Gastos del día” solo lista los gastos de la fecha activa y no duplica tarjetas.

• El FAB `+` abre el flujo de alta sin navegación rota.

• La barra inferior glass no tapa contenido ni el FAB y mantiene iconografía coherente.

• No quedan textos explicativos innecesarios ni “Movimiento manual” compitiendo con el importe.

• Sin errores visibles al no tener presupuesto, no tener gastos, superar el presupuesto o introducir importes límite razonables.

12. Preguntas que el repositorio debe resolver

No se han recuperado con certeza del historial, así que Codex no debe inventarlas:

• ¿Qué librería concreta implementa el store global?

• ¿Qué mecanismo exacto de persistencia local está instalado?

• ¿Cuál es la versión efectiva de Expo/RN en el branch actual?

• ¿Expo Router sustituyó completamente la navegación previa?

• ¿Qué componentes y nombres de archivos existen ya para Home, gastos y presupuesto?

• ¿Qué parte del Calendario/Compartidos/Resumen diario está ya codificada?

• ¿Cuál es el estado real del APK/build Android más reciente?

• ¿Qué assets actuales son los definitivos para icono y splash?

13. Prompt de arranque recomendado para Codex

> Estás continuando WEEKASH. Antes de escribir código, inspecciona el repositorio completo. La autoridad es: (1) código/package.json/lockfile, (2) ARQUITECTURA.md, (3) Weekash_Handoff_Codex_Contexto_Completo.pdf, (4) FINANZAS_APP.md y app antigua solo como referencias. No inventes que una función está implementada. Clasifica el estado real por fases: Base, Home, Pulido, Calendario, Compartidos, Ajustes, QA y Build Android. Respeta la arquitectura existente y no migres dependencias sin necesidad. Prioridad inmediata: identificar la fase activa y terminarla. Para Home, conserva el enfoque funcional y limpio: presupuesto semanal editable con lápiz verde, dinero restante como foco, “Has usado X de Y”, “Semana actual” L-D, pills modernas, una única tarjeta “Gastos del día”, FAB +, navegación inferior glass con iconos verdes y umbrales de gasto 0-49 verde / 50-74 amarillo / 75-89 naranja / 90+ rojo. Primero entrégame un diagnóstico del repo y un plan mínimo de cambios. Después implementa solo lo necesario para cerrar la fase activa, ejecuta las comprobaciones disponibles y resume archivos modificados, decisiones y pendientes.

14. Nota sobre fidelidad del documento

Este handoff consolida información recuperada de las conversaciones de Weekash y contexto técnico conservado del proyecto. No contiene una copia del repositorio ni permite afirmar el estado exacto de cada archivo hoy. Por eso, cualquier detalle marcado como CONTEXTO COMPLETADO debe contrastarse con el código antes de aplicarlo. La intención es preservar decisiones y reducir al mínimo la pérdida de contexto entre ChatGPT y Codex.